

Backtracking

From "Generate Subsets" to N-Queens and Sudoku -- A Derive-First Study Guide

Built from a Socratic ChatGPT study session (132 pages). Every solution is tested against independent brute-force enumeration. Every diagram traces an actual run of the code shown next to it.

How to read this guide

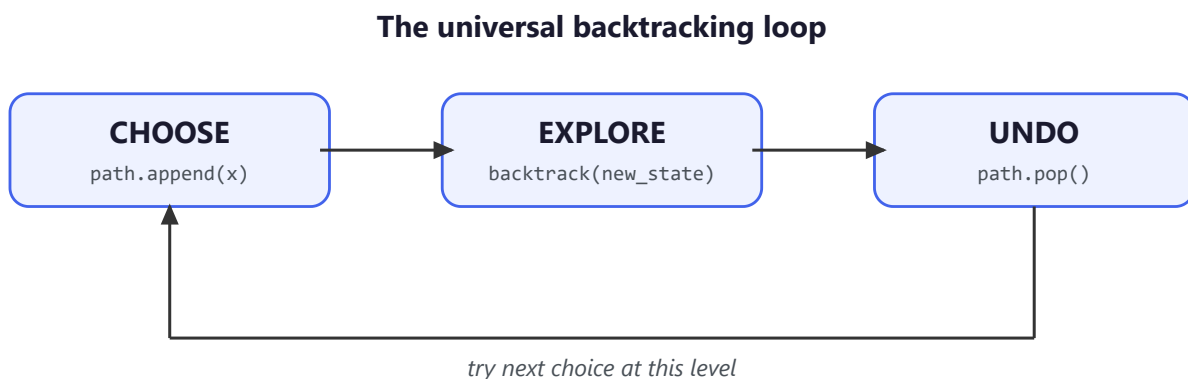
This guide does not hand you templates to memorize. It rebuilds backtracking from a single mental model -- **choose -> explore -> undo** -- and shows how every classic problem (Subsets, Permutations, N-Queens, Sudoku, Word Search, partitioning problems...) is the *same* loop with a different **state**, **choice set**, and **pruning rule**.

Each pattern section follows the same shape:

1. **The decision** -- what question are you asking at each node of the tree?
2. **State, choices, base case, undo** -- the five-question checklist applied.
3. **Derivation** -- how the code falls out of the decision tree, step by step.
4. **Tested code** -- the exact function verified in `test_sol.py`.
5. **Trace** -- a real step-by-step run, not a hand-wavy example.
6. **Recognition trigger** -- the phrase/shape that should make you think "this pattern."

A dedicated **Correction** callout (red) preserves a real moment from the source chat where an early answer was wrong and had to be fixed -- these are usually the most valuable moments, because the *reasoning* behind the fix is exactly the kind of thing interviewers probe for.

Roadmap



Part	Topic	Core new idea
1	Fundamentals	choose/explore/undo, backtracking vs. DFS vs. recursion
2	Subsets & Subsets II	take/skip, same-level duplicate pruning
3	Combinations family	<code>start</code> pointer, reuse (<code>i</code> vs <code>i+1</code>), the source chat's correction
4	Permutations family	<code>used[]</code> , duplicate-aware permutations, tile counts
5	Constrained sequences	Parentheses, N-Queens (I/II), Sudoku
6	Grid backtracking	Word Search (I/II), Path with Maximum Gold
7	Partitioning	Palindrome Partitioning, Restore IP, Word Break II
8	Extension problems	Combination Sum III, Target Sum, K-subsets, and more
9	Complexity	branching factor x depth, how pruning changes the bound
10	Decision guide	the five pattern shapes, recognition drills
11	Interview Q&A	the questions this material actually prepares you for

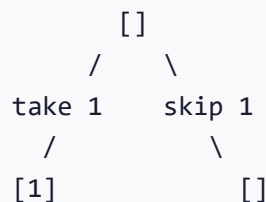
Part 1 -- Fundamentals

1.1 The core idea

Picture a problem where you make a *sequence of choices*. For `nums = [1, 2, 3]`, generating every subset:

```
[]
```

You encounter `1`. Two choices: **take 1** or **skip 1**. That single decision already gives you a tree:



Do the same for `2`, then `3`, and the whole search space is a tree where **every node is a partial solution**. `[1,2]` doesn't mean "done" -- it means "so far, I've chosen 1 and 2." That's the entire mental model.

1.2 Where "backtracking" comes in

Suppose you're at `[1,2]` and you've finished exploring everything below it. To try `[1,3]` next, you must first **undo** the choice of `2`:

```
choose 1 -> choose 2 -> explore everything with [1,2] -> UNDO 2 -> choose 3 ->
explore [1,3]
```

That's the shape:

```
CHOOSE -> EXPLORE -> UNDO -> NEXT CHOICE
```

The undo matters because the *same* `path` object is reused across branches. The invariant to protect at all times:

`path` represents exactly the choices made along the **current** recursion path -- nothing more, nothing left over from a sibling branch.

Concretely:

```
choose 1      path = [1]
choose 2      path = [1,2]
finish branch
undo 2        path = [1]
choose 3      path = [1,3]
```

Skip the undo and you'd incorrectly end up with `[1,2,3]` while believing you're exploring `[1,3]`.

1.3 Backtracking = DFS over choices (but not every DFS is backtracking)

You can view the whole decision tree as a graph being traversed via DFS: go down one branch as far as possible, then climb back and try the next branch.

```
[] -> [1] -> [1,2] -> [1,2,3]
                        |
                        undo 3
                        |
                        [1,2] -- try the other branch
```

But be careful. Ordinary tree traversal (`visit node -> visit left -> visit right`) doesn't construct a candidate solution and undo choices -- that's DFS without backtracking. Backtracking is specifically DFS **plus** the choose -> explore -> undo structure over a space of decisions.

Recursion vs. DFS vs. backtracking, precisely:

- **Recursion:** a function calls itself. That's it.
- **DFS:** explore nodes of a graph/tree, going deep before wide.
- **Backtracking:** DFS/recursion over *choices*, where you mutate shared state (a path, a visited set, a partially-filled board) before recursing and **restore it** after, so the next sibling branch starts clean.

Every backtracking solution uses recursion/DFS-like exploration, but not every recursive function or DFS is backtracking.

1.4 The universal five-question checklist

Before writing a single line of code on a new problem, answer these in order:

1. **STATE** -- what completely describes where I am in the decision tree?
2. **CHOICES** -- from this state, what can I choose next?
3. **CONSTRAINT** -- which choices are illegal? (this is where pruning lives)
4. **BASE CASE** -- when is the current partial solution complete?
5. **UNDO** -- what did I mutate before recursing that must be restored after?

```
def backtrack(state):
    if base_case:
        record_answer()
        return
    for choice in choices(state):
        if invalid(choice):
            continue
        apply(choice)          # CHOOSE
        backtrack(new_state)    # EXPLORE
        undo(choice)            # UNDO
```

Don't start from this skeleton. Derive state -> choices -> constraints -> base case -> undo *first*; the skeleton is what falls out afterward. That derivation is exactly what Parts 2-8 walk through for every classic pattern.

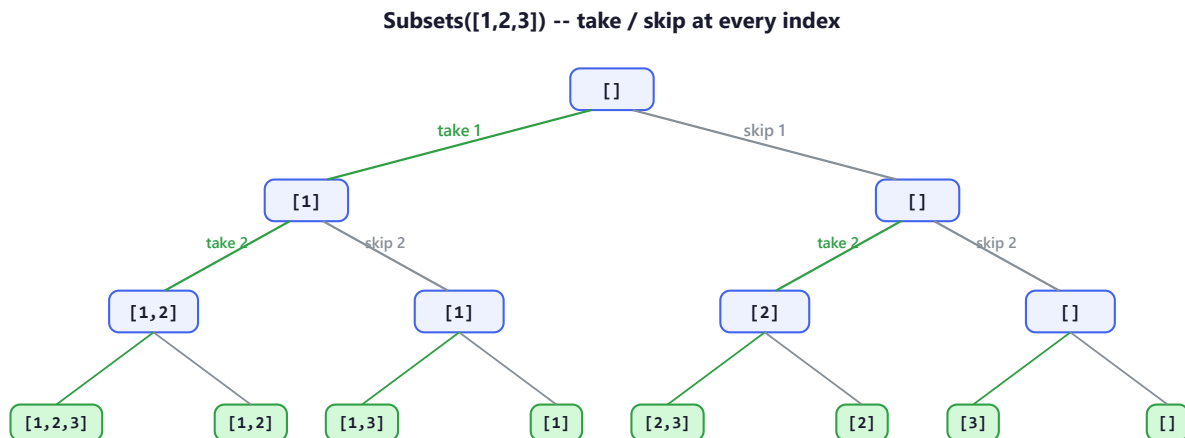
1.5 How to recognize backtracking in an interview

Strong signal phrases: "*generate all...*", "*find all possible...*", "*return all combinations/permutations...*", "*all valid...*", "*all ways...*", "*partition...*", "*arrange...*", "*place...*" -- especially when the count of possibilities is exponential in the input size. When you hear one of these, reach for the five-question checklist before reaching for code.

Part 2 -- Subsets (take / skip)

2.1 The decision

"Generate every possible subset" is the canonical **binary decision** pattern: at every element, you either take it or skip it.



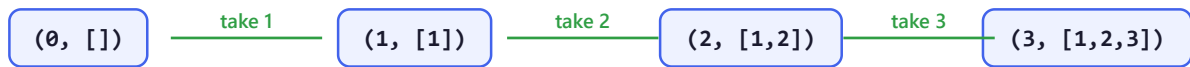
Every root-to-leaf path is one subset. $n=3 \rightarrow 2^3 = 8$ leaves.

2.2 State, choices, base case, undo

Question	Answer
State	<code>index</code> (where am I) + <code>path</code> (what have I chosen)
Choices	take <code>nums[index]</code> , or skip <code>nums[index]</code>
Constraint	none -- every element is always legal to take or skip
Base case	<code>index == len(nums)</code>
Undo	<code>path.pop()</code> after the take branch

A common point of confusion: `path = [1, 2]` alone doesn't tell you whether `3` has already been processed. Compare `path=[1,2], index=2` against `path=[1,2], index=3` -- same path, different points in the tree. `index` is what disambiguates them.

State = (index, path) as recursion descends



index -> WHERE am I? path -> WHAT have I chosen?

Base case: `index == len(nums) -> res.append(path.copy())`

2.3 Derivation

Starting from `index=0, path=[]`, the **take** branch appends the element, recurses, then pops it back off before trying **skip**:

```
MAKE CHOICE      path.append(nums[index])
EXPLORE          backtrack(index + 1)
UNDO CHOICE      path.pop()
(then the skip branch just recurses again -- path is already restored)
```

Notice something subtle: after `pop()`, `path` is already back to its pre-take state, so the skip branch needs **no extra undo of its own** -- it simply never touched `path` in the first place.

At the base case, one detail matters a lot:

Should you append `path` itself, or a **copy** of `path`?

Since `path` is a single mutable list being repeatedly `append` ed and `pop` ped, storing the same object reference means every later mutation would retroactively corrupt every previously recorded answer. You must store `path.copy()`.

2.4 Tested code

```
def subsets(nums: List[int]) -> List[List[int]]:
    res = []
    path = []

    def backtrack(index):
        if index == len(nums):
            res.append(path.copy())
            return
        # Take nums[index]
        path.append(nums[index])
        backtrack(index + 1)
        path.pop()
        # Skip nums[index]
        backtrack(index + 1)

    backtrack(0)
    return res
```

2.5 Trace: subsets([1, 2, 3])

Index	Path	Action
0	[]	
0	[1]	take 1
1	[1]	
1	[1, 2]	take 2
2	[1, 2]	
2	[1, 2, 3]	take 3
3	[1, 2, 3]	BASE -> record
2	[1, 2]	undo (pop 3)
2	[1, 2]	skip 3
3	[1, 2]	BASE -> record

Index	Path	Action
1	[1]	undo (pop 2)
1	[1]	skip 2
2	[1]	
2	[1, 3]	take 3
3	[1, 3]	BASE -> record
2	[1]	undo (pop 3)
2	[1]	skip 3
3	[1]	BASE -> record
0	[]	undo (pop 1)
0	[]	skip 1
1	[]	
1	[2]	take 2
2	[2]	
2	[2, 3]	take 3
3	[2, 3]	BASE -> record
2	[2]	undo (pop 3)
2	[2]	skip 3
3	[2]	BASE -> record
1	[]	undo (pop 2)
1	[]	skip 2
2	[]	
2	[3]	take 3
3	[3]	BASE -> record
2	[]	undo (pop 3)
2	[]	skip 3
3	[]	BASE -> record

Every root-to-leaf path in the tree corresponds to exactly one subset. With 2 choices at each of 3 elements: $2 \times 2 \times 2 = 2^3 = 8$ subsets.

2.6 Subsets II -- duplicates in the input

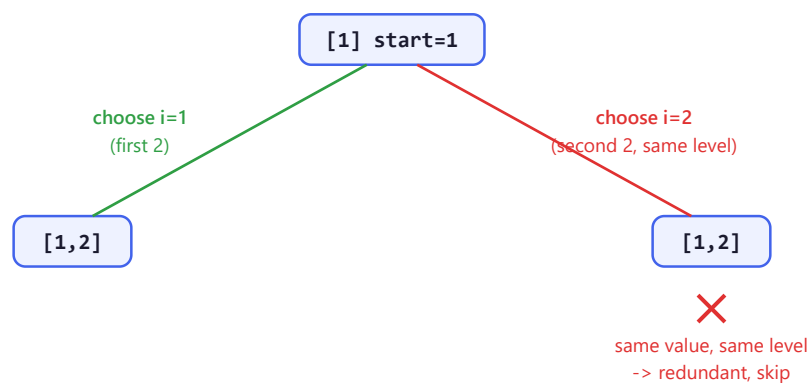
Now suppose `nums = [1, 2, 2]` and you want **unique** subsets. Without duplicate handling, the tree treats the two `2`s as distinguishable choices and can produce the same subset twice: *take the first 2, skip the second and skip the first 2, take the second both yield `[2]`*.

The key insight: duplicates are only redundant when they are **alternative choices at the same recursion level**.

```
[2, 2] <- choosing the first 2 vs. the second 2 at the SAME level
           leads to identical subsequent possibilities -> redundant, skip one
```

```
[2]
|
[2,2] <- choosing the second 2 at a DEEPER level (after already
           taking the first) is completely valid -- not a duplicate branch
```

Subsets II [1,2,2] -- skip a duplicate CHOICE at the same level



```
nums.sort(); if i > start and nums[i] == nums[i-1]: continue
```

(deeper-level repeats, e.g. choosing 2 then 2 again, are still allowed)

So the rule is:

```
same level + same value -> redundant -> skip
deeper level + same value -> potentially meaningful -> allow
```

To make "same value at the same level" detectable, **sort first** so equal values become adjacent, then guard with `i > start and nums[i] == nums[i-1]`.

2.7 Tested code -- Subsets II

```
def subsets_with_dup(nums: List[int]) -> List[List[int]]:
    nums = sorted(nums)
    res = []
    path = []

    def backtrack(start):
        res.append(path.copy())
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
    return res
```

This version is written in the equivalent **loop form** (`start` + a `for` loop over remaining indices) rather than explicit take/skip recursive calls. Both forms explore the identical tree for the take/skip pattern -- the loop form is simply more convenient once you need a per-level duplicate guard, and it's the form you'll reuse immediately in Combinations next.

2.8 Recognition trigger

"All subsets / subsequences / every possible selection" -> think `take / skip`, state = `index + path`. Add "duplicates in input" + "unique subsets"
-> think `sort` + `skip duplicate choices at the same level`.

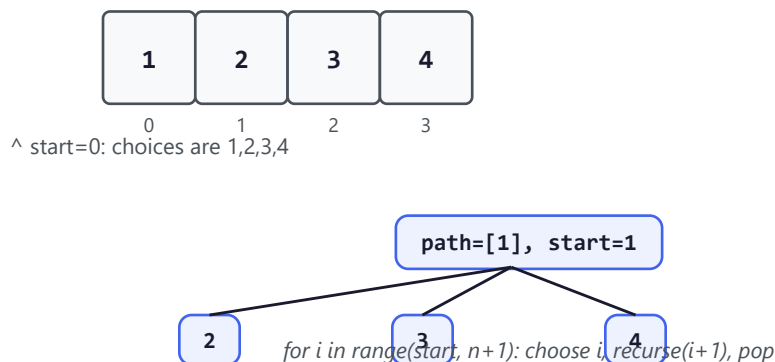
Part 3 -- Combinations, Combination Sum, Combination Sum II

3.1 A different decision: "which element should I choose next?"

Subsets asks "take this specific element or not?" at every element. Combinations asks a different question entirely: "which of the remaining elements should I pick?" This is **pattern B -- choose from remaining**, and it needs a different piece of state.

For `nums=[1,2,3,4]`, `k=2`, once `path=[1]`, you must never go backward and choose `1` again -- combinations don't care about order (`[1,2] == [2,1]`), so re-choosing an earlier index would just generate the same combination twice, or worse, create `[1,2,1]` which isn't a combination at all.

Combinations: start = 'which positions may I still choose from'



3.2 index vs. start -- the distinction that looks identical but isn't

Both are integers pointing into the array, which is exactly why they're easy to confuse. The difference is **what the number means**:

	Subsets: index	Combinations: start
Meaning	"I'm deciding about <i>this one</i> element"	"I may choose <i>any</i> element from here on"
Decision	take <code>nums[index]</code> / skip <code>nums[index]</code>	<code>for i in range(start, n)</code>
After a choice	always advances to <code>index + 1</code>	advances to <code>i + 1</code> (the <i>chosen</i> index, not <code>start</code>)

SUBSETS

index=1

[1, 2, 3]
^

two choices:

take 2 / skip 2

COMBINATIONS

start=1

[1, 2, 3]
^^^^

all of these are choices:

choose 2, choose 3

One sentence to remember: **index** = which element am I deciding about? **start** = from which position can I choose my next element?

3.3 State, choices, base case, undo -- Combinations

Question	Answer
State	<code>start + path</code>
Choices	any <code>i</code> in <code>range(start, n+1)</code>
Base case	<code>len(path) == k</code>
Undo	<code>path.pop()</code> , then the loop naturally advances to the next <code>i</code>
No reuse	recurse with <code>backtrack(i + 1)</code> -- never revisit an earlier position

3.4 Tested code -- Combinations

```
def combine(n: int, k: int) -> List[List[int]]:
    res = []
    path = []

    def backtrack(start):
        if len(path) == k:
            res.append(path.copy())
            return
        for i in range(start, n + 1):
            path.append(i)
            backtrack(i + 1)
            path.pop()

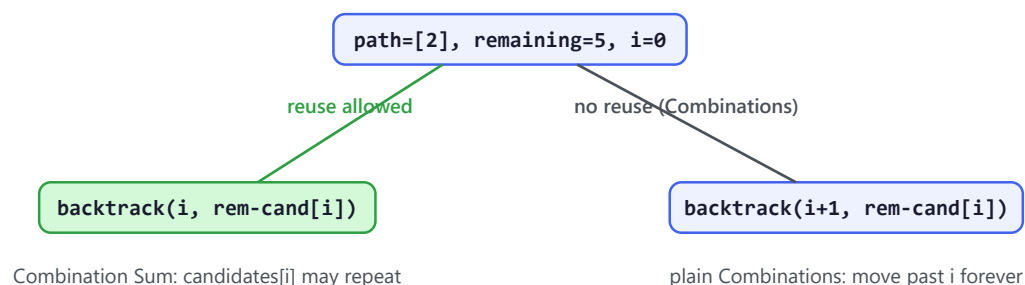
    backtrack(1)
    return res
```

3.5 Combination Sum -- introducing reuse

Now the problem changes: `candidates=[2,3,6,7]`, `target=7`, find every combination summing to the target, and **the same candidate may be reused**. If you choose `2`, you're still allowed to choose `2` again: `[2] -> [2,2] -> [2,2,2] -> ...`

This is where the single most important implementation detail in this whole family shows up: whether the recursive call reuses index `i` or advances to `i + 1`.

Combination Sum -- can the current candidate be reused?



Rule: can I choose the SAME element again right now? yes -> i no -> i+1

can reuse the current choice again?

|

+- YES -> backtrack(i, ...) (Combination Sum)

|

+- NO -> backtrack(i + 1, ...) (plain Combinations)

Correction, worked from the source chat

While deriving Combination Sum, the chat asked: "we're currently at candidate 3 (index 1). If we choose 3, what's the next index? If we skip 3, what's the next index?"

The first answer given was **reversed**: choose -> index 2, skip -> index 1.

That's backwards, and it's a genuinely common interview bug, not a typo.

Why the reversed version breaks:

- If **choosing** 3 advances the index (`i+1`), you've just forbidden reusing 3, even though the problem explicitly allows unlimited reuse. Any answer requiring 3 twice (or 2 three times, etc.) silently disappears from the output -- with no error, no exception, just a wrong (incomplete) result set.
- If **skipping** 3 keeps the index unchanged (`i`), the loop revisits the same candidate every time it's skipped, in the worst case never making forward progress.

The correct rule, re-derived properly:

- **Choose** 3 -> next index = 1 (same `i`) -- 3 remains available because reuse is allowed.
- **Skip** 3 -> next index = 2 (`i+1`) -- once you decide never to use 3 on this branch, move past it permanently.

Correction from the source chat: don't swap the choose/skip index rule

candidates=[2,3,6,7], target=7, currently deciding about candidate 3 (i=1)

WRONG (student's first answer, reversed)

```
choose 3 -> next index = 2 (i+1)
skip 3 -> next index = 1 (i)
```

Effect if coded this way:

- choosing 3 permanently forbids reusing 3, even though reuse IS allowed
- "skipping" 3 keeps revisiting index 1 forever (no progress)



misses [2,2,3]-style reuse; can hang

CORRECT (after the correction)

```
choose 3 -> next index = 1 (i)
skip 3 -> next index = 2 (i+1)
```

Why:

- choosing 3 means it's still available next call, since reuse is allowed -> i
- skipping 3 means "never use 3 again" -> move past it -> i+1

```
backtrack(i, rem-cand[i])
```

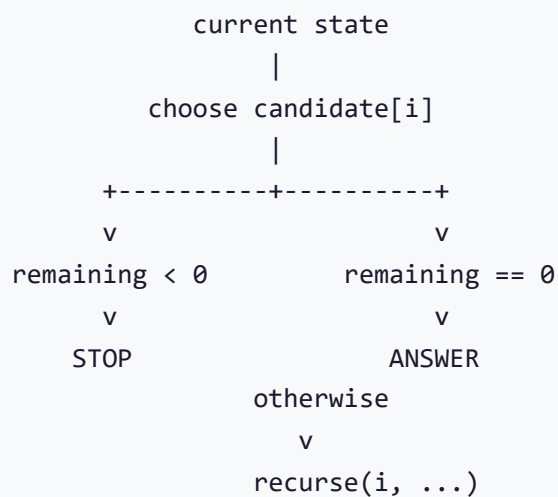
reaches [2,2,3], remaining=0 -> recorded

Mental check: reuse allowed -> the choice branch stays at i; only the loop's next i moves on.

`test_sol.py` 's `combination_sum` tests would have caught this immediately: for `target=6` the correct solver finds `[2,2,2]` , `[3,3]` , `[6]` ; a solver that always recurses with `i+1` finds only `[3,3]` and `[6]` -- the reuse-dependent answer silently vanishes. This is exactly the kind of bug that produces *plausible-looking, quietly incomplete* output, which is why tracing the index rule by hand (rather than pattern-matching on `i` vs `i+1` from memory) matters.

3.6 State, choices, base case, undo -- Combination Sum

Question	Answer
State	<code>start</code> + <code>remaining</code> + <code>path</code>
Choices	<code>candidates[i]</code> for <code>i</code> in <code>range(start, len(candidates))</code>
Constraint (prune)	<code>remaining < 0</code> -- this branch can never succeed, stop immediately
Base case	<code>remaining == 0</code>
Reuse rule	<code>backtrack(i, remaining - candidates[i])</code> -- not <code>i + 1</code>



3.7 Tested code -- Combination Sum

```

def combination_sum(candidates: List[int], target: int) -> List[List[int]]:
    res = []
    path = []

    def backtrack(start, remaining):
        if remaining == 0:
            res.append(path.copy())
            return
        if remaining < 0:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, remaining - candidates[i]) # i: reuse allowed
            path.pop()

    backtrack(0, target)
    return res

```

3.8 Trace: combination_sum([2,3,6,7], 7)

i	Path	Remaining	Action
0	[2]	5	choose 2, recurse(i=0)
0	[2, 2]	3	choose 2, recurse(i=0)
0	[2, 2, 2]	1	choose 2, recurse(i=0)
0	[2, 2, 2, 2]	-1	choose 2, recurse(i=0)
0	[2, 2, 2, 2]	-1	remaining<0 -> prune
1	[2, 2, 2, 3]	-2	choose 3, recurse(i=1)
1	[2, 2, 2, 3]	-2	remaining<0 -> prune
2	[2, 2, 2, 6]	-5	choose 6, recurse(i=2)
2	[2, 2, 2, 6]	-5	remaining<0 -> prune
3	[2, 2, 2, 7]	-6	choose 7, recurse(i=3)
3	[2, 2, 2, 7]	-6	remaining<0 -> prune
1	[2, 2, 3]	0	choose 3, recurse(i=1)

i	Path	Remaining	Action
1	[2, 2, 3]	0	remaining==0 -> record
2	[2, 2, 6]	-3	choose 6, recurse(i=2)
2	[2, 2, 6]	-3	remaining<0 -> prune
3	[2, 2, 7]	-4	choose 7, recurse(i=3)
3	[2, 2, 7]	-4	remaining<0 -> prune
1	[2, 3]	2	choose 3, recurse(i=1)
1	[2, 3, 3]	-1	choose 3, recurse(i=1)
1	[2, 3, 3]	-1	remaining<0 -> prune
2	[2, 3, 6]	-4	choose 6, recurse(i=2)
2	[2, 3, 6]	-4	remaining<0 -> prune
3	[2, 3, 7]	-5	choose 7, recurse(i=3)
3	[2, 3, 7]	-5	remaining<0 -> prune
2	[2, 6]	-1	choose 6, recurse(i=2)
2	[2, 6]	-1	remaining<0 -> prune
3	[2, 7]	-2	choose 7, recurse(i=3)
3	[2, 7]	-2	remaining<0 -> prune
1	[3]	4	choose 3, recurse(i=1)
1	[3, 3]	1	choose 3, recurse(i=1)
1	[3, 3, 3]	-2	choose 3, recurse(i=1)
1	[3, 3, 3]	-2	remaining<0 -> prune
2	[3, 3, 6]	-5	choose 6, recurse(i=2)
2	[3, 3, 6]	-5	remaining<0 -> prune
3	[3, 3, 7]	-6	choose 7, recurse(i=3)
3	[3, 3, 7]	-6	remaining<0 -> prune
2	[3, 6]	-2	choose 6, recurse(i=2)
2	[3, 6]	-2	remaining<0 -> prune

i	Path	Remaining	Action
3	[3, 7]	-3	choose 7, recurse(i=3)
3	[3, 7]	-3	remaining<0 -> prune
2	[6]	1	choose 6, recurse(i=2)
2	[6, 6]	-5	choose 6, recurse(i=2)
2	[6, 6]	-5	remaining<0 -> prune
3	[6, 7]	-6	choose 7, recurse(i=3)
3	[6, 7]	-6	remaining<0 -> prune
3	[7]	0	choose 7, recurse(i=3)
3	[7]	0	remaining==0 -> record

3.9 Combination Sum II -- no reuse, plus duplicates

Combine everything so far: `candidates=[10,1,2,7,6,1,5]`, `target=8`, each number usable **at most once**, and duplicate combinations forbidden.

This is not a new pattern -- it's three already-derived rules stacked:

Combination (no reuse: `i+1`) + duplicates-in-input (sort + same-level skip) + target constraint (prune)

Combination Sum II: sort first, then skip dup + break early



```

if i > start and candidates[i] == candidates[i-1]: continue
if candidates[i] > remaining: break # sorted -> rest are too big too

```

Combines: no-reuse (`i+1`) + duplicate-skip (sort + same-level guard) + prune (break)

```

candidates.sort()
for i in range(start, len(candidates)):
    if i > start and candidates[i] == candidates[i - 1]:
        continue                # same-level duplicate guard
    if candidates[i] > remaining:
        break                    # sorted -> everything after is too
                                # big too
    ...
    backtrack(i + 1, remaining - candidates[i])    # no reuse

```

3.10 Tested code -- Combination Sum II

```

def combination_sum2(candidates: List[int], target: int) -> List[List[int]]:
    candidates = sorted(candidates)
    res = []
    path = []

    def backtrack(start, remaining):
        if remaining == 0:
            res.append(path.copy())
            return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue
            if candidates[i] > remaining:
                break            # sorted -> everything after is too big too
            path.append(candidates[i])
            backtrack(i + 1, remaining - candidates[i])    # i+1: no reuse
            path.pop()

    backtrack(0, target)
    return res

```

3.11 Recognition trigger

"Choose k elements, order doesn't matter" -> `start + path`, `i + 1` (no reuse). "...and the same element can be reused" -> keep `start`, but recurse

with `i` (not `i+1`), and prune on `remaining < 0`. "...and the input has duplicates, no reuse" -> sort + same-level duplicate guard + `i + 1`.

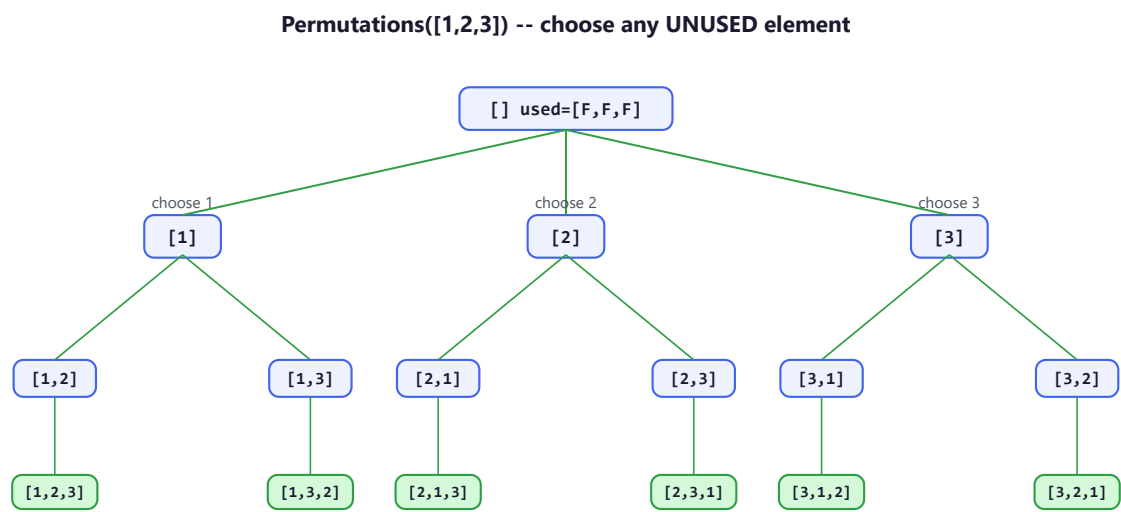
Part 4 -- Permutations, Permutations II, Letter Tile Possibilities

4.1 A third kind of decision

Subsets asks "take this element or not?" Combinations asks "which remaining element should I choose?" Permutations asks yet another question:

"Which currently-unused element goes into the next position?"

Unlike Combinations, **order matters** and there's no forward-only `start` -- any unused element can go next, including ones with a smaller index than what you've already placed.



State = path + used[]. Base case: len(path) == len(nums). 3! = 6 leaves.

4.2 State, choices, base case, undo

Question	Answer
State	<code>path</code> + <code>used[]</code>
Choices	every index <code>i</code> where <code>used[i]</code> is <code>False</code>

Question	Answer
Base case	<code>len(path) == len(nums)</code>
Undo	<code>path.pop()</code> and <code>used[i] = False</code> -- two mutations, two undos

`used[i]` is the piece of state that makes this pattern different from Subsets: it's not about position in the array, it's about which *values* have already been placed on the current branch.

4.3 Tested code

```
def permute(nums: List[int]) -> List[List[int]]:
    res = []
    path = []
    used = [False] * len(nums)

    def backtrack():
        if len(path) == len(nums):
            res.append(path.copy())
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return res
```

4.4 Trace: permute([1, 2])

Path	Used	Action
[1]	[True, False]	choose nums[0]=1

Path	Used	Action
[1, 2]	[True, True]	choose nums[1]=2
[1, 2]	[True, True]	BASE -> record
[2]	[False, True]	choose nums[1]=2
[2, 1]	[True, True]	choose nums[0]=1
[2, 1]	[True, True]	BASE -> record

4.5 Permutations II -- duplicate values

`nums=[1,1,2]` should produce 3 distinct permutations, not $3! = 6$. Sort first, then apply the same "same value + same level -> redundant" rule from Subsets II, adapted for the used-array structure: skip index `i` if it's a duplicate of `i-1` **and** `i-1` hasn't been used on this branch yet (meaning we'd be trying the two identical values in the same relative order twice).

```

def permute_unique(nums: List[int]) -> List[List[int]]:
    nums = sorted(nums)
    res = []
    path = []
    used = [False] * len(nums)

    def backtrack():
        if len(path) == len(nums):
            res.append(path.copy())
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue # duplicate at this level -> skip
            used[i] = True
            path.append(nums[i])
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return res

```

The `not used[i-1]` guard is the subtle part. It specifically blocks placing the *second* copy of a repeated value before the *first* copy has been used anywhere on the current path -- which is exactly the situation that produces a duplicate permutation. If the first copy is already used deeper in the path, placing the second copy now is a different (valid, non-duplicate) arrangement.

4.6 Letter Tile Possibilities -- counting instead of indices

`tiles="AAB"` : generate every non-empty arrangement. This *looks* like Permutations, but tracking `used[i]` per character position would overcount, since the two `A`s are interchangeable. The chat's fix: track **counts per letter value** instead of a `used[]` array per position.

```
count[i] -= 1
backtrack()
count[i] += 1
```

Because identical letters share one counter, choosing "the A" is a single branch instead of two indistinguishable ones -- no duplicate-skip condition needed at all; the redundant branches simply never exist.

4.7 Tested code

```
def num_tile_possibilities(tiles: str) -> int:
    count = [0] * 26
    for ch in tiles:
        count[ord(ch) - ord("A")] += 1
    result = 0

    def backtrack():
        nonlocal result
        for i in range(26):
            if count[i] == 0:
                continue
            result += 1
            count[i] -= 1
            backtrack()
            count[i] += 1

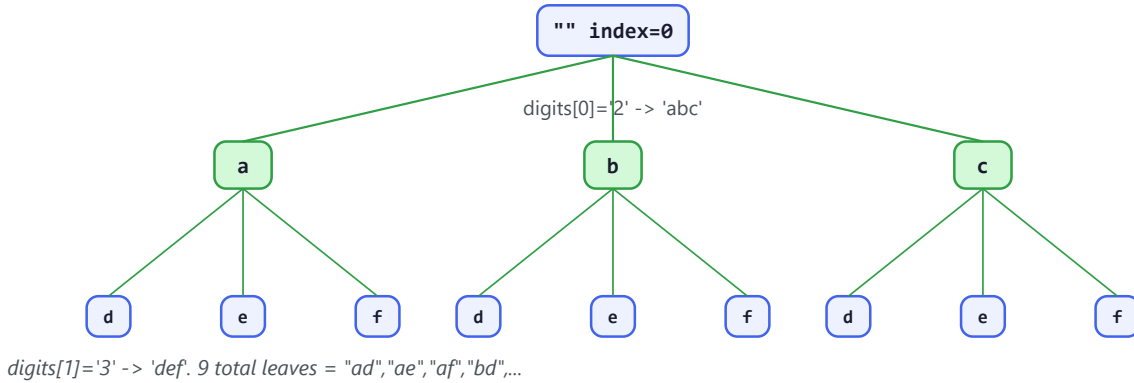
    backtrack()
    return result
```

4.8 Letter Combinations of a Phone Number -- one choice per position, from a fixed set

A short but distinct pattern worth placing here: `digits="23"` with the classic phone-keypad mapping (`2->abc` , `3->def` , ...). At each position the choices are fixed by that position's digit -- structurally this is `index + path` exactly like Subsets, except instead

of a binary take/skip decision, every position offers 3-4 choices drawn from a lookup table.

Letter Combinations("23") -- one choice per digit position



Question	Answer
State	<code>index</code> + <code>path</code>
Choices	every letter in <code>mapping[digits[index]]</code>
Base case	<code>index == len(digits)</code>
Undo	<code>path.pop()</code> after each letter

```

def letter_combinations(digits: str) -> List[str]:
    if not digits:
        return []
    mapping = {
        "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
        "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz",
    }
    res = []
    path = []

    def backtrack(index):
        if index == len(digits):
            res.append("".join(path))
            return
        for ch in mapping[digits[index]]:
            path.append(ch)
            backtrack(index + 1)
            path.pop()

    backtrack(0)
    return res

```

4.9 Recognition trigger

"Arrange / reorder all elements" -> permutation -> state = `path + used[]`,
 choose any unused element. "...with duplicate values" -> sort + `not`
`used[i-1]` guard, **or** switch to counting distinct values instead of tracking
 positions.

Part 5 -- Constrained Sequences: Parentheses, N-Queens, Sudoku

5.1 A shift in what "pruning" means

Subsets/Combinations/Permutations prune only for *redundancy* (duplicates). This family prunes for **validity** -- a choice can be flatly illegal given what's already been decided, and you reject it before ever recursing.

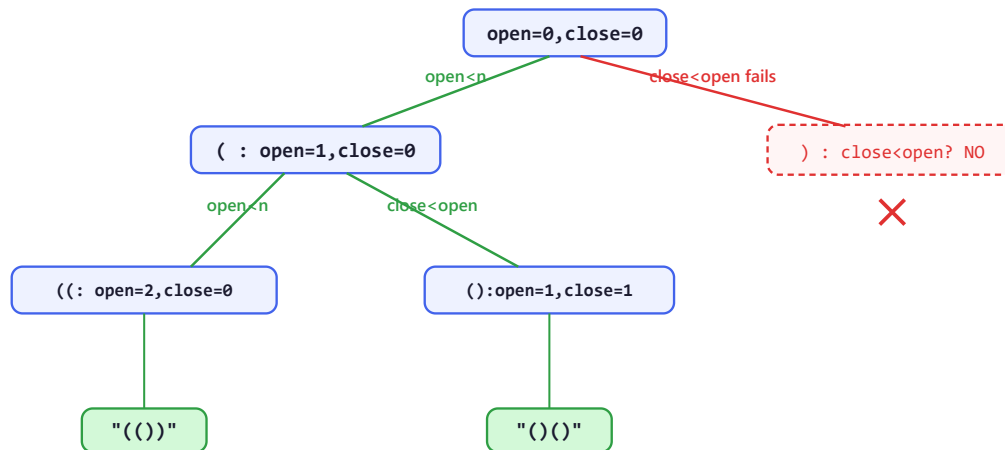
5.2 Generate Parentheses -- constraint-based choices

For `n=3`, build strings with exactly 3 `(` and 3 `)` that are always balanced. State: `(open, close, path)`. At every position there are potentially 2 choices, but not always both:

```
can add '(' only while open < n
can add ')' only while close < open
```

Why the second rule? If `open=2, close=2`, adding `)` gives `(()))` -- already invalid, no matter what follows. This is pruning **before** it happens, not detecting it after the fact.

Generate Parentheses(n=2) -- constrained choices, invalid branch pruned



Rules: add '(' only if $open < n$. Add ')' only if $close < open$ (never create an invalid prefix).

Question	Answer
State	<code>open</code> + <code>close</code> + <code>path</code>
Choices	add <code>(</code> , add <code>)</code>
Constraint	<code>(</code> only if <code>open < n</code> ; <code>)</code> only if <code>close < open</code>
Base case	<code>len(path) == 2*n</code>
Undo	<code>path.pop()</code> after each branch

5.3 Tested code -- Generate Parentheses

```

def generate_parenthesis(n: int) -> List[str]:
    res = []
    path = []

    def backtrack(open_count, close_count):
        if len(path) == 2 * n:
            res.append("".join(path))
            return
        if open_count < n:
            path.append("(")
            backtrack(open_count + 1, close_count)
            path.pop()
        if close_count < open_count:
            path.append(")")
            backtrack(open_count, close_count + 1)
            path.pop()

    backtrack(0, 0)
    return res

```

5.4 Trace: generate_parenthesis(2)

Open	Close	Path	Action
1	0	(	add (
2	0	((	add (
2	1	(()	add)
2	2	(())	add)
2	2	(())	len==2n -> record
1	1	()	add)
2	1	()(	add (
2	2	()()	add)
2	2	()()	len==2n -> record

5.5 N-Queens -- constraint checking gets stronger

Place one queen per row so no two attack each other. The decision at each row: *which column?* Three ways a placement can conflict with an existing queen:

- 1. same column
- 2. same `\` diagonal
- 3. same `/` diagonal

The diagonal trick, derived rather than memorized: look at cells on a `\` diagonal -- `(0,0),(1,1),(2,2),(3,3)` -- and notice `row - col` is constant (`0`) for all of them. On a `/` diagonal -- `(0,3),(1,2),(2,1),(3,0)` -- `row + col` is constant (`3`). So three sets (`cols`, `diag1 = row-col`, `diag2 = row+col`) fully capture "is this placement safe?" in $O(1)$ per check.

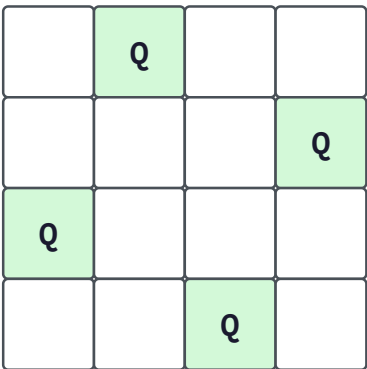
N-Queens: same column, or row-col, or row+col -> conflict

col:
same column already has a queen

row - col:
constant along a `\` diagonal

row + col:
constant along a `/` diagonal

if col in cols or
row-col in diag1 or
row+col in diag2:
continue # reject



A valid 4-Queens solution

Question	Answer
State	<code>row + cols + diag1 + diag2</code> (+ the board itself for output)
Choices	every <code>col</code> in <code>range(n)</code>
Constraint	<code>col in cols or row-col in diag1 or row+col in diag2</code>
Base case	<code>row == n</code>
Undo	remove <code>col</code> from <code>cols</code> , <code>row-col</code> from <code>diag1</code> , <code>row+col</code> from <code>diag2</code>

5.6 Tested code -- N-Queens (and N-Queens II)

```
def solve_n_queens(n: int) -> List[List[str]]:
    res = []
    board = [ "." * n for _ in range(n)]
    cols = set()
    diag1 = set() # row - col
    diag2 = set() # row + col

    def backtrack(row):
        if row == n:
            res.append(["".join(r) for r in board])
            return
        for col in range(n):
            if col in cols or row - col in diag1 or row + col in diag2:
                continue
            board[row][col] = "Q"
            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            backtrack(row + 1)
            board[row][col] = "."
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

    backtrack(0)
    return res
```

N-Queens II only needs a **count** of solutions, not the boards themselves -- same state, same pruning, the base case increments a counter instead of recording a board:

```

def total_n_queens(n: int) -> int:
    cols = set()
    diag1 = set()
    diag2 = set()
    count = 0

    def backtrack(row):
        nonlocal count
        if row == n:
            count += 1
            return
        for col in range(n):
            if col in cols or row - col in diag1 or row + col in diag2:
                continue
            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            backtrack(row + 1)
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

    backtrack(0)
    return count

```

5.7 Trace: solve_n_queens(4), first solution found

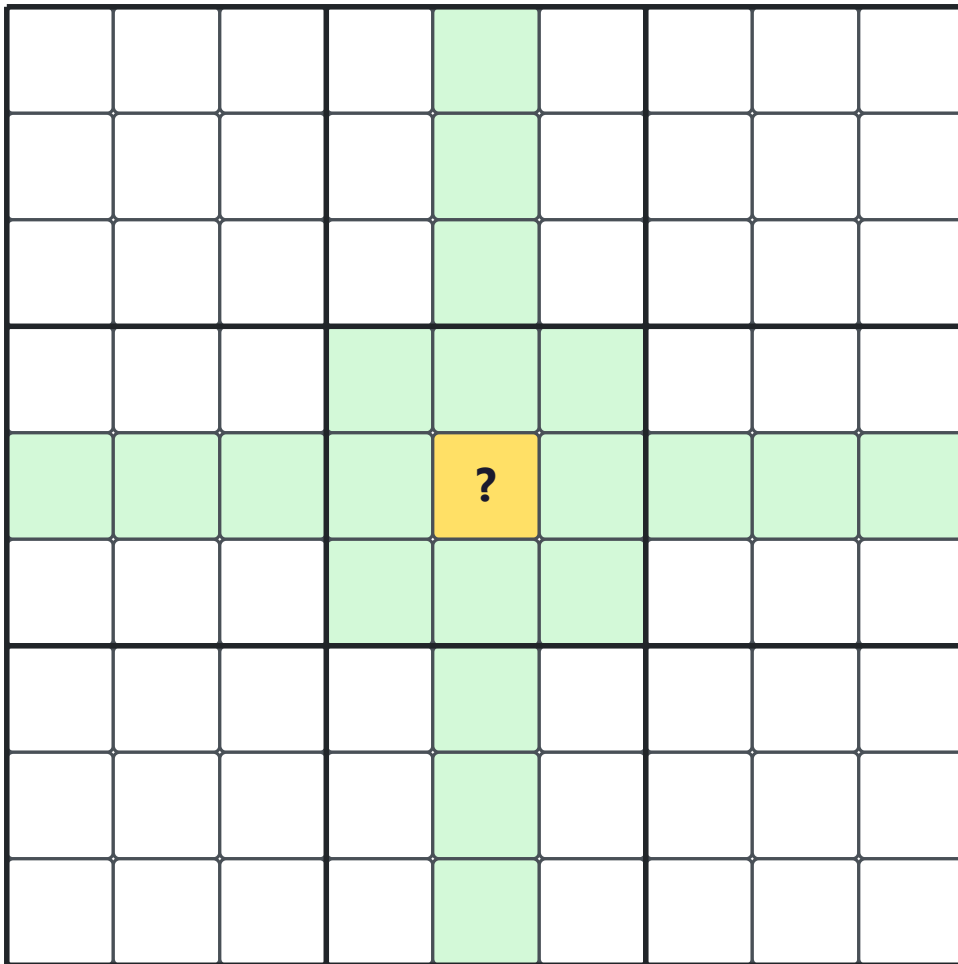
Row	Col	Action
0	0	place queen
1	0	reject: column/diagonal conflict
1	1	reject: column/diagonal conflict
1	2	place queen
2	0	reject: column/diagonal conflict
2	1	reject: column/diagonal conflict
2	2	reject: column/diagonal conflict

Row	Col	Action
2	3	reject: column/diagonal conflict
1	2	undo (backtrack)
1	3	place queen
2	0	reject: column/diagonal conflict
2	1	place queen
3	0	reject: column/diagonal conflict
3	1	reject: column/diagonal conflict
3	2	reject: column/diagonal conflict
3	3	reject: column/diagonal conflict
2	1	undo (backtrack)
2	2	reject: column/diagonal conflict
2	3	reject: column/diagonal conflict
1	3	undo (backtrack)
0	0	undo (backtrack)
0	1	place queen
1	0	reject: column/diagonal conflict
1	1	reject: column/diagonal conflict
1	2	reject: column/diagonal conflict
1	3	place queen
2	0	place queen
3	0	reject: column/diagonal conflict
3	1	reject: column/diagonal conflict
3	2	place queen

5.8 Sudoku Solver -- the same pattern, a bigger constraint set

9x9 grid, some cells fixed, others .. For each empty cell, try digits 1 - 9, rejecting any digit already present in that row, column, or 3x3 box.

Sudoku: a candidate must be absent from its row, column, and 3x3 box



Highlighted = every cell that could block a candidate at (4,4)

Notice the structural similarity to N-Queens:

```
N-Queens:  choose column  -> check constraints -> recurse -> undo
Sudoku:    choose number  -> check constraints -> recurse -> undo
```

The *object being placed* changed (a queen vs. a digit); the backtracking skeleton didn't. One implementation wrinkle worth internalizing: Sudoku's `backtrack()` returns a boolean (found a full solution or not) so the very first success can propagate all the

way back up and stop the search -- otherwise you'd keep exploring after already solving the puzzle.

Question	Answer
State	current empty cell (found by scanning) + the board
Choices	digits 1 - 9
Constraint	not already in this row, column, or 3x3 box
Base case	no empty cell remains -> solved
Undo	reset the cell back to . if no digit led to a full solution

5.9 Tested code -- Sudoku Solver

```

def solve_sudoku(board: List[List[str]]) -> None:
    def is_valid(r, c, num):
        for i in range(9):
            if board[r][i] == num:
                return False
        for i in range(9):
            if board[i][c] == num:
                return False
        box_r, box_c = (r // 3) * 3, (c // 3) * 3
        for i in range(box_r, box_r + 3):
            for j in range(box_c, box_c + 3):
                if board[i][j] == num:
                    return False
        return True

    def backtrack():
        for r in range(9):
            for c in range(9):
                if board[r][c] != ".":
                    continue
                for num in "123456789":
                    if not is_valid(r, c, num):
                        continue
                    board[r][c] = num
                    if backtrack():
                        return True
                    board[r][c] = "."
                return False
        return True

    backtrack()

```

5.10 Recognition trigger

"Build a sequence/board under validity rules" -> constrained backtracking.
 Ask: *"can I reject this branch immediately, before recursing?"* That question is the entire difference between this family and brute-force generate-then-filter.

Part 6 -- Grid Backtracking: Word Search, Word Search II, Path with Maximum Gold

6.1 A new flavor of choice: movement

So far every "choice" has been an array index or a character. Word Search's choice is a **movement** in a 2D state space -- up, down, left, right -- and the thing you mutate-then-undo is a `visited` set rather than a `path` list.

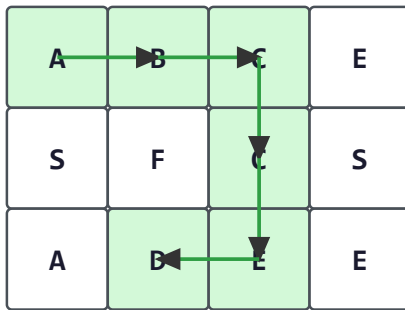
```
board = [ ["A", "B", "C", "E"], ["S", "F", "C", "S"], ["A", "D", "E", "E"] ], word = "ABCCED" .
```

6.2 State, choices, base case, undo

Question	Answer
State	<code>(row, col, word_index) + visited</code>
Choices	move up / down / left / right
Constraint	in-bounds, not already visited, <code>board[nr][nc] == word[index]</code>
Base case	<code>index == len(word)</code> -> return <code>True</code> (not a copy of the path!)
Undo	<code>visited.remove((r, c))</code> after exploring all 4 directions

An early wrong instinct here is to think the base case should "return the path," mirroring Subsets/Permutations. But Word Search only needs a **yes/no** answer -- does *any* valid path exist? -- so the base case returns `True` and lets that boolean propagate back up through the `or` chain, rather than collecting complete paths into `res`. This is a meaningfully different shape from every pattern in Parts 2-4.

Word Search: board find "ABCCED" -- move, mark visited, undo on failure



`visited.add((r,c)) -> explore 4 dirs -> visited.remove((r,c)) on backtrack`

Checks before moving to (nr,nc): in-bounds, not visited, `board[nr][nc]==word[index]`

The overall shape:

```
check current cell matches word[index]
|
mark visited
|
try 4 directions
|
any succeeds? -- yes --> True
|
no
|
unmark visited (undo)
|
False
```

That unmark-on-failure is what allows the **same cell** to be reused by a different candidate path later in the search.

6.3 Tested code -- Word Search

```

def exist(board: List[List[str]], word: str) -> bool:
    rows = len(board)
    cols = len(board[0])
    visited = set()

    def backtrack(r, c, index):
        if index == len(word):
            return True
        if (
            r < 0 or r >= rows or
            c < 0 or c >= cols or
            (r, c) in visited or
            board[r][c] != word[index]
        ):
            return False
        visited.add((r, c))
        found = (
            backtrack(r + 1, c, index + 1) or
            backtrack(r - 1, c, index + 1) or
            backtrack(r, c + 1, index + 1) or
            backtrack(r, c - 1, index + 1)
        )
        visited.remove((r, c))
        return found

    for r in range(rows):
        for c in range(cols):
            if backtrack(r, c, 0):
                return True
    return False

```

6.4 Trace: exist(board, "ABCCED")

Row	Col	Index	Action
0	0	0	visit 'A' matches word[0]
0	1	1	visit 'B' matches word[1]
0	2	2	visit 'C' matches word[2]

Row	Col	Index	Action
1	2	3	visit 'C' matches word[3]
2	2	4	visit 'E' matches word[4]
2	1	5	visit 'D' matches word[5]

6.5 Word Search II -- many words at once (Trie + backtracking)

Searching the whole board separately for each word in a dictionary is wasteful -- overlapping prefixes get re-explored from scratch every time. Building a **Trie** of all target words first lets a single grid DFS check many words simultaneously: at each cell, only descend into the Trie if the current character is a valid next step for *some* word, and whenever a Trie node marks the end of a word, record it.

```

choose direction -> is board[r][c] a child of the current Trie node?
    |
    yes -> descend Trie, mark cell used, recurse
    |
    Trie node ends a word? -> record it
    |
    undo: unmark cell

```

6.6 Tested code -- Word Search II

```

def find_words(board: List[List[str]], words: List[str]) -> List[str]:
    trie = {}
    END = "$"
    for word in words:
        node = trie
        for ch in word:
            node = node.setdefault(ch, {})
        node[END] = word

    rows, cols = len(board), len(board[0])
    found = set()

    def backtrack(r, c, node):
        ch = board[r][c]
        if ch not in node:
            return
        nxt = node[ch]
        if END in nxt:
            found.add(nxt[END])
        board[r][c] = "#"
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != "#":
                backtrack(nr, nc, nxt)
        board[r][c] = ch

    for r in range(rows):
        for c in range(cols):
            backtrack(r, c, trie)
    return list(found)

```

6.7 Path with Maximum Gold -- undo that enables revisiting, not exclusion

A grid backtracking variant with a twist: instead of Word Search's "never revisit a cell in this path," Path with Maximum Gold zeroes out a cell's value while visiting it (so it can't be double-counted within one path) and restores it afterward -- meaning the **same cell can be the start of a different exploration later**, since the mutation is local to one DFS call's lifetime, not a persistent global `visited` set.

```

def get_maximum_gold(grid: List[List[int]]) -> int:
    rows, cols = len(grid), len(grid[0])
    best = 0

    def backtrack(r, c):
        nonlocal best
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == 0:
            return 0
        gold = grid[r][c]
        grid[r][c] = 0 # mark visited (this IS the undo-able mutation)
        total = gold + max(
            backtrack(r + 1, c), backtrack(r - 1, c),
            backtrack(r, c + 1), backtrack(r, c - 1),
        )
        grid[r][c] = gold # undo
        best = max(best, total)
        return total

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != 0:
                backtrack(r, c)
    return best

```

6.8 Recognition trigger

"Grid + form a path + can't reuse a cell within that path" -> DFS + `visited.add()/remove()` around a 4-directional (or 8-directional) move set. If the answer is boolean ("does a path exist"), propagate `True` up through `or`; if the answer is "all paths," collect into `res` at the base case, same as Subsets.

Part 7 -- Partitioning: Palindrome Partitioning, Restore IP Addresses, Word Break II

7.1 The generalization: "where does the next piece end?"

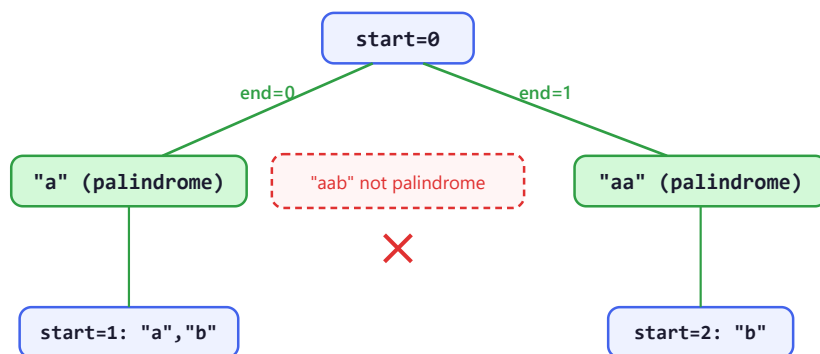
This is presented in the source material as a genuinely useful generalization once you've internalized Combinations' `start` pointer:

When a problem asks for all ways to **partition** something, the choice at each step is usually *"where does the next piece end?"*

7.2 Palindrome Partitioning

`s = "aab"` -> `[["a", "a", "b"], ["aa", "b"]]`. At `start`, try every possible end position for the next piece, but only accept it if `s[start:end+1]` is a palindrome.

Palindrome Partitioning("aab") -- choose where the next piece ends



Result: `[["a", "a", "b"], ["aa", "b"]]`. Base case: `start == len(s)`.

Question	Answer
State	<code>start</code> + <code>path</code>

Question	Answer
Choices	<code>end</code> in <code>range(start, len(s))</code>
Constraint	<code>s[start:end+1]</code> must be a palindrome
Base case	<code>start == len(s)</code>
Undo	<code>path.pop()</code>

7.3 Tested code

```
def partition(s: str) -> List[List[str]]:
    res = []
    path = []

    def is_palindrome(left, right):
        while left < right:
            if s[left] != s[right]:
                return False
            left += 1
            right -= 1
        return True

    def backtrack(start):
        if start == len(s):
            res.append(path.copy())
            return
        for end in range(start, len(s)):
            if not is_palindrome(start, end):
                continue
            path.append(s[start:end + 1])
            backtrack(end + 1)
            path.pop()

    backtrack(0)
    return res
```

7.4 Trace: partition("aab")

Start	Path	Action
1	['a']	choose 'a'
2	['a', 'a']	choose 'a'
3	['a', 'a', 'b']	choose 'b'
3	['a', 'a', 'b']	start==len(s) -> record
1	['a']	'ab' not palindrome -> skip
2	['aa']	choose 'aa'
3	['aa', 'b']	choose 'b'
3	['aa', 'b']	start==len(s) -> record
0	[]	'aab' not palindrome -> skip

7.5 Restore IP Addresses -- partition into exactly 4 valid pieces

"25525511135" -> "255.255.11.135", "255.255.111.35", ... Same "where does the next piece end?" question, but now bounded to segments of length 1-3, with two extra validity rules: no leading zero (unless the segment is exactly "0"), and numeric value ≤ 255 . The base case fires only once **both** conditions hold: exactly 4 segments **and** every character consumed.

```

if len(path) == 4:
    if start == len(s):
        res.append(".".join(path))
    return
for end in range(start, min(len(s), start + 3)):
    part = s[start:end + 1]
    if len(part) > 1 and part[0] == "0":
        continue
    if int(part) > 255:
        continue
    ...

```

7.6 Tested code

```
def restore_ip_addresses(s: str) -> List[str]:
    res = []
    path = []

    def backtrack(start):
        if len(path) == 4:
            if start == len(s):
                res.append(".".join(path))
            return
        for end in range(start, min(len(s), start + 3)):
            part = s[start:end + 1]
            if len(part) > 1 and part[0] == "0":
                continue
            if int(part) > 255:
                continue
            path.append(part)
            backtrack(end + 1)
            path.pop()

    backtrack(0)
    return res
```

7.7 Word Break II -- partition against a dictionary constraint

`s = "catsanddog"`, `wordDict = ["cat", "cats", "and", "sand", "dog"]` -> `"cat sand dog"`, `"cats and dog"`. Identical partition shape; the constraint is simply "is this substring in the dictionary?" instead of "is this substring a palindrome?" -- reinforcing that the *pattern* (partition: choose where the next piece ends) is what to recognize, not the specific validity check.

```

def word_break_ii(s: str, word_dict: List[str]) -> List[str]:
    word_set = set(word_dict)
    res = []
    path = []

    def backtrack(start):
        if start == len(s):
            res.append(" ".join(path))
            return
        for end in range(start + 1, len(s) + 1):
            word = s[start:end]
            if word not in word_set:
                continue
            path.append(word)
            backtrack(end)
            path.pop()

    backtrack(0)
    return res

```

7.8 Recognition trigger

"Partition a string/sequence into valid pieces, return all ways" -> state = `start + path`, choices = every possible next-piece end position, constraint = whatever makes a piece "valid" for this problem (palindrome, dictionary word, numeric range...), base case = `start == len(s)`.

Part 8 -- Extension Problems

These weren't walked through in the source chat, but every one of them is a direct application of a pattern already derived in Parts 2-7. They're included so the guide's problem pool is fully self-contained, and every solution below is verified in `test_sol.py` against an independent brute-force check.

8.1 Combination Sum III -- Combinations + a sum target + fixed count

Choose exactly `k` numbers from `1..9` (each used at most once) summing to `n`. This is `combine()`'s "no reuse" index rule (`i+1`) plus `combination_sum`'s remaining-sum tracking, with a stronger prune: since the range `1..9` is already sorted, `if i > remaining: break` cuts a branch the moment even the smallest step would overshoot.

```
def combination_sum3(k: int, n: int) -> List[List[int]]:
    res = []
    path = []

    def backtrack(start, remaining):
        if len(path) == k:
            if remaining == 0:
                res.append(path.copy())
            return
        for i in range(start, 10):
            if i > remaining:
                break
            path.append(i)
            backtrack(i + 1, remaining - i)
            path.pop()

    backtrack(1, n)
    return res
```

8.2 Target Sum -- backtracking as a clean way to see a DP problem

Assign `+` or `-` to each number in `nums` so the total equals `target`. Every number gets exactly 2 choices (sign), and there's no explicit `path` -- just a running total threaded through the recursion. This is Pattern A (binary decision) in a different costume: instead of "take/skip an element into a set," it's "add/subtract an element into a running sum."

```
def find_target_sum_ways(nums: List[int], target: int) -> int:
    count = 0

    def backtrack(index, total):
        nonlocal count
        if index == len(nums):
            if total == target:
                count += 1
            return
        backtrack(index + 1, total + nums[index])
        backtrack(index + 1, total - nums[index])

    backtrack(0, 0)
    return count
```

8.3 Partition to K Equal Sum Subsets

Split `nums` into `k` groups of equal sum. State = which elements are used + current group's running sum + how many groups are already complete. Two prunes make this tractable: sort descending (fail fast on oversized elements) and skip a duplicate value if its identical predecessor was *not* used -- the same same-level-duplicate idea from Subsets II, applied to "which element starts a new group."

```

def can_partition_k_subsets(nums: List[int], k: int) -> bool:
    total = sum(nums)
    if total % k != 0:
        return False
    target = total // k
    nums = sorted(nums, reverse=True)
    if nums[0] > target:
        return False
    n = len(nums)
    used = [False] * n

    def backtrack(groups_formed, current_sum, start_index):
        if groups_formed == k:
            return True
        if current_sum == target:
            return backtrack(groups_formed + 1, 0, 0)
        for i in range(start_index, n):
            if used[i] or current_sum + nums[i] > target:
                continue
            # skip duplicate starts at the same recursion level
            if i > start_index and nums[i] == nums[i - 1] and not used[i - 1]:
                continue
            used[i] = True
            if backtrack(groups_formed, current_sum + nums[i], i + 1):
                return True
            used[i] = False
        return False

    return backtrack(0, 0, 0)

```

8.4 Beautiful Arrangement

Build a permutation of `1..n` where, at each position `i`, the placed value `perm[i]` satisfies `perm[i] % i == 0 or i % perm[i] == 0`. Structurally identical to Permutations (`used[]`, choose any unused value) with one constraint check added before each choice is committed.

```

def count_arrangement(n: int) -> int:
    used = [False] * (n + 1)
    count = 0

    def backtrack(position):
        nonlocal count
        if position > n:
            count += 1
            return
        for num in range(1, n + 1):
            if used[num]:
                continue
            if num % position == 0 or position % num == 0:
                used[num] = True
                backtrack(position + 1)
                used[num] = False

    backtrack(1)
    return count

```

8.5 Word Pattern II -- backtracking over a bijective mapping

Does `s` match `pattern` under some **bijection** between pattern characters and substrings of `s`? The choice at each step is "how long is the next substring assigned to this pattern character?" -- a partition-style decision -- constrained so no two pattern characters can map to the same substring (hence two maps, `char_to_word` and `word_to_char`, both updated and undone together).

```

def word_pattern_match(pattern: str, s: str) -> bool:
    char_to_word = {}
    word_to_char = {}

    def backtrack(pi, si):
        if pi == len(pattern) and si == len(s):
            return True
        if pi == len(pattern) or si == len(s):
            return False
        ch = pattern[pi]
        if ch in char_to_word:
            word = char_to_word[ch]
            if s.startswith(word, si):
                return backtrack(pi + 1, si + len(word))
            return False
        for end in range(si + 1, len(s) + 1):
            word = s[si:end]
            if word in word_to_char:
                continue
            char_to_word[ch] = word
            word_to_char[word] = ch
            if backtrack(pi + 1, end):
                return True
            del char_to_word[ch]
            del word_to_char[word]
        return False

    return backtrack(0, 0)

```

8.6 Generalized Abbreviation

Generate every valid abbreviation of a word (e.g. "word" -> "word", "1ord", "w1rd", ..., "4"). Binary decision at each character: abbreviate it (extend a numeric run) or keep it literally (which forces flushing any pending run first). The subtlety is entirely in correctly opening/closing that numeric run at the right moments.

```

def generate_abbreviations(word: str) -> List[str]:
    res = []
    path = []

    def backtrack(index, run_length):
        if index == len(word):
            if run_length > 0:
                path.append(str(run_length))
                res.append("".join(path))
            if run_length > 0:
                path.pop()
            return
        # Option 1: abbreviate word[index] (extend the numeric run)
        backtrack(index + 1, run_length + 1)
        # Option 2: keep word[index] literally (must flush any pending run)
        if run_length > 0:
            path.append(str(run_length))
            path.append(word[index])
            backtrack(index + 1, 0)
            path.pop()
        if run_length > 0:
            path.pop()

    backtrack(0, 0)
    return res

```

8.7 Palindrome Permutation II

Generate every distinct palindromic permutation of `s`. Only strings with at most one odd-count character can form a palindrome at all; once that's confirmed, you only need to permute the **first half** of the character multiset (duplicate-aware, same `not used[i-1]` guard as Permutations II) and mirror it around an optional single middle character.

```

def generate_palindrome_permutations(s: str) -> List[str]:
    from collections import Counter
    counts = Counter(s)
    odd_chars = [ch for ch, c in counts.items() if c % 2 == 1]
    if len(odd_chars) > 1:
        return []
    middle = odd_chars[0] if odd_chars else ""
    half_counts = {ch: c // 2 for ch, c in counts.items()}
    half_chars = []
    for ch, c in half_counts.items():
        half_chars.extend([ch] * c)

    res = []
    path = []
    used = [False] * len(half_chars)
    half_chars.sort()

    def backtrack():
        if len(path) == len(half_chars):
            half = "".join(path)
            res.append(half + middle + half[::-1])
            return
        for i in range(len(half_chars)):
            if used[i]:
                continue
            if i > 0 and half_chars[i] == half_chars[i - 1] and not used[i - 1]:
                continue
            used[i] = True
            path.append(half_chars[i])
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return res

```

8.8 Binary Watch, Numbers With Same Consecutive Differences, Find Unique Binary String, Sequential Digits

Four short, self-contained backtracking problems that are easy to underestimate as "just enumerate" -- each one is still choose -> explore -> undo over a small, well-defined choice set:

- **Binary Watch:** choose which LEDs are lit (10 binary choices), base case = exactly `turned_on` LEDs lit and the resulting time is valid.
- **Numbers With Same Consecutive Differences:** choose the next digit from `{last+k, last-k}`, pruned to stay in `0..9`.
- **Find Unique Binary String:** build a length-`n` binary string bit by bit, backtracking out of any prefix that's a dead end (in practice a direct diagonalization argument is faster, but the backtracking framing generalizes to inputs where diagonalization doesn't obviously apply).
- **Sequential Digits:** build increasing runs of consecutive digits (`123` , `234` , ..., `789`), filtering to the `[low, high]` range.

```
def read_binary_watch(turned_on: int) -> List[str]:
    res = []

    def backtrack(index, count, hour, minute):
        if count == turned_on:
            if hour < 12 and minute < 60:
                res.append(f"{hour}:{minute:02d}")
            return
        if index == 10 or count > turned_on:
            return
        # LEDs 0-3 are hours (values 8,4,2,1), LEDs 4-9 are minutes (32..1)
        if index < 4:
            value = [8, 4, 2, 1][index]
            backtrack(index + 1, count + 1, hour + value, minute)
        else:
            value = [32, 16, 8, 4, 2, 1][index - 4]
            backtrack(index + 1, count + 1, hour, minute + value)
            backtrack(index + 1, count, hour, minute)

    backtrack(0, 0, 0, 0)
    return res
```

```

def nums_same_consec_diff(n: int, k: int) -> List[int]:
    res = []
    path = []

    def backtrack():
        if len(path) == n:
            res.append(int("".join(map(str, path))))
            return
        last = path[-1]
        nxts = {last + k, last - k}
        for nxt in nxts:
            if 0 <= nxt <= 9:
                path.append(nxt)
                backtrack()
                path.pop()

    for start in range(1, 10):
        path.append(start)
        backtrack()
        path.pop()

    return res

```

```

def find_different_binary_string(nums: List[str]) -> str:
    n = len(nums)
    seen = set(nums)
    path = []

    def backtrack():
        if len(path) == n:
            candidate = "".join(path)
            return candidate if candidate not in seen else None
        for bit in "01":
            path.append(bit)
            result = backtrack()
            path.pop()
            if result is not None:
                return result
        return None

    return backtrack()

```

```

def sequential_digits(low: int, high: int) -> List[int]:
    res = []
    digits = "123456789"

    def backtrack(start_index, num):
        value = int(num)
        if value > high:
            return
        if value >= low and len(num) >= 2:
            res.append(value)
        if start_index + 1 <= len(digits):
            return
        backtrack(start_index + 1, num + digits[start_index + 1])

    for i in range(len(digits)):
        backtrack(i, digits[i])
    return sorted(res)

```

8.9 Matchsticks to Square, Sum of All Subset XOR Totals

- **Matchsticks to Square:** partition sticks into 4 equal-sum groups -- Partition-to-K-Subsets specialized to `k=4`, with a symmetry prune (`if sides[i] == sides[i-1]: continue`) that skips trying an identical partial side twice.
- **Sum of All Subset XOR Totals:** pure take/skip (Pattern A, identical shape to Subsets) accumulating an XOR instead of appending to a list.

```

def makesquare(matchsticks: List[int]) -> bool:
    total = sum(matchsticks)
    if total % 4 != 0:
        return False
    side = total // 4
    matchsticks = sorted(matchsticks, reverse=True)
    if matchsticks and matchsticks[0] > side:
        return False
    sides = [0, 0, 0, 0]

    def backtrack(index):
        if index == len(matchsticks):
            return sides[0] == sides[1] == sides[2] == side
        for i in range(4):
            if sides[i] + matchsticks[index] > side:
                continue
            if i > 0 and sides[i] == sides[i - 1]:
                continue # symmetry pruning: identical partial sides
            sides[i] += matchsticks[index]
            if backtrack(index + 1):
                return True
            sides[i] -= matchsticks[index]
        return False

    return backtrack(0)

```

```

def subset_xor_sum(nums: List[int]) -> int:
    total = 0

    def backtrack(index, current_xor):
        nonlocal total
        if index == len(nums):
            total += current_xor
            return
        backtrack(index + 1, current_xor ^ nums[index]) # take
        backtrack(index + 1, current_xor) # skip

    backtrack(0, 0)
    return total

```

8.10 Recognition trigger

None of Part 8 introduces a *new* shape. The exercise here is transfer: given a problem you haven't seen, map it onto Parts 2-7's five shapes (binary decision / choose-from-remaining / constrained sequence / grid movement / partition) before writing any code.

Part 9 -- Complexity: Branching Factor x Depth

9.1 The shape of the bound

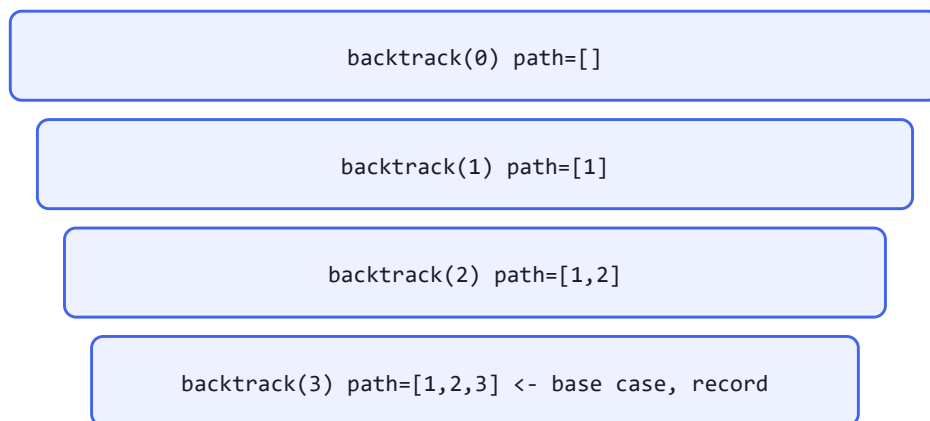
Every backtracking search explores (a subset of) a tree. The two numbers that determine its size:

- **branching factor** b -- how many choices exist at each node
- **depth** d -- how many decisions are made along one root-to-leaf path

Without any pruning, the tree has on the order of b^d nodes. Pruning doesn't change this *worst-case* bound (a problem can always be constructed where no branch gets cut), but it changes the **typical/expected** runtime enormously, which is why N-Queens or Sudoku with constraint checks are fast in practice despite an exponential bound on paper.

9.2 Per-pattern branching factor and depth

Call-stack depth while exploring [1,2,3] down the 'take all' branch



Stack unwinds: path.pop() after each return -> depth mirrors index

Pattern	Branching factor	Depth	Bound (loose)
Subsets	2 (take/skip)	<code>n</code>	<code>O(2^n)</code>
Subsets II	<code><= 2</code> , pruned by dedup	<code>n</code>	<code>O(2^n)</code> worst case, less in practice
Combinations (<code>n</code> choose <code>k</code>)	shrinks as <code>start</code> advances	<code>k</code>	<code>O(C(n,k) * k)</code> (the <code>*k</code> is the copy cost)
Combination Sum	up to <code>len(candidates)</code> , reuse allowed	up to <code>target / min(candidates)</code>	exponential in <code>target</code>
Permutations	shrinks as elements get used (<code>n</code> , <code>n-1</code> , <code>n-2</code> , ...)	<code>n</code>	<code>O(n!)</code>
Generate Parentheses	<code><= 2</code>	<code>2n</code>	bounded tightly by the n-th Catalan number, <code>O(4^n / n^1.5)</code> -- far below <code>2^(2n)</code> thanks to the <code>close < open</code> prune
Word Search	<code><= 4</code> (grid moves)	<code>len(word)</code>	<code>O(rows * cols * 4^L)</code>
N-Queens	shrinks per row via pruning	<code>n</code>	naive <code>O(n^n)</code> ; pruning brings real runs down close to <code>O(n!)</code>
Sudoku	<code><= 9</code> per empty cell	<code><= 81</code>	huge naive bound; row/col/box pruning is what makes it solvable in practice
Partitioning (Palindrome, IP, Word Break)	up to <code>len(s) - start</code>	up to <code>len(s)</code>	exponential in <code>len(s)</code> , trimmed hard by the validity check

9.3 Why the `.copy()` matters for complexity, not just correctness

Every base case that does `res.append(path.copy())` pays `O(depth)` to copy the path. Across the whole tree this adds a `* d` factor on top of the node count -- which

is why the "bound" column above is described as loose: the *number of leaves* is the dominant term, but each leaf's own output costs something too (a string join, a board copy, a path copy).

9.4 Pruning is a constant-factor-looking change that isn't

It's tempting to read "pruning" as a minor optimization. In problems like N-Queens or Sudoku, pruning is the difference between *tractable* and *not-finishing-in-your-lifetime* for realistic input sizes, because it cuts off exponentially many descendants at once -- rejecting a choice at depth k removes an entire $b^{(d-k)}$ -sized subtree, not just one node. The earlier (shallower) a prune fires, the more it saves.

9.5 Space complexity

Backtracking's space cost is almost always dominated by:

1. **Recursion depth** -- $O(d)$ stack frames.
2. **The mutable state being threaded through** -- `path`, `visited`, `used[]`, or a board -- typically $O(d)$ as well, since it grows and shrinks with the recursion.
3. **The output**, if you're collecting all solutions -- this can dominate everything else (e.g. $O(2^n * n)$ for Subsets, since there are 2^n subsets each up to length n).

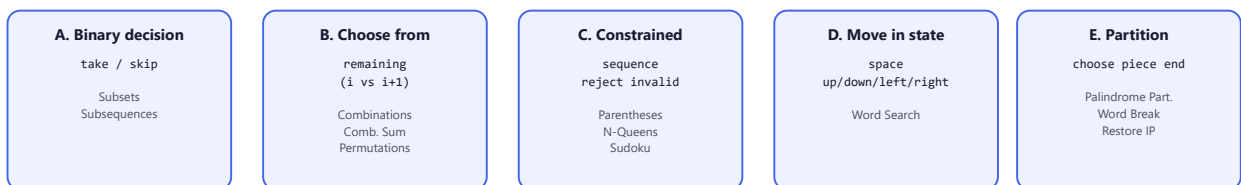
The in-place mutate/undo discipline (rather than passing a new copy of `path` / `board` down every call) is precisely what keeps (1) and (2) from each becoming $O(d^2)$ or worse.

Part 10 -- Decision Guide

10.1 The five decision-tree shapes

You don't actually have 20+ unrelated algorithms in this guide. You have five recurring shapes:

Five decision-tree shapes behind (almost) every backtracking problem



Ask: state? choices? valid? done? what to undo?

Shape	Question at each node	Examples
A. Binary decision	Take this element, or skip it?	Subsets, Subset XOR Sum, Target Sum
B. Choose from remaining	Which of the remaining options do I pick? (reuse: i vs i+1)	Combinations, Combination Sum (I/II/III), Permutations
C. Constrained sequence	Is this choice even legal right now?	Generate Parentheses, N-Queens, Sudoku, Beautiful Arrangement
D. Move in state space	Which neighboring cell/state do I move to?	Word Search (I/II), Path with Maximum Gold
E. Partition	Where does the next piece end?	Palindrome Partitioning, Restore IP, Word Break II, Word Pattern II

10.2 State cheat sheet

State definitions across the pattern family

Subsets	index + path
Subsets II	index/start + path (+ sort)
Combinations	start + path
Combination Sum	start + remaining + path
Combination Sum II	start + remaining + path (+ sort)
Permutations	path + used[]
Letter Combinations	index + path
Generate Parentheses	open + close + path
Word Search	row + col + index + visited
N-Queens	row + cols/diag1/diag2
Palindrome Partition	start + path
Sudoku Solver	cell + board

10.3 The most important recognition difference

Three patterns look deceptively similar on first read -- all three iterate over "the array" -- but ask fundamentally different questions:

Subsets	"Should I take THIS element?"	-> take / skip
Combinations	"Which remaining element do I choose?"	-> for i in range(start, n)
Permutations	"Which UNUSED element goes here?"	-> for i in range(n): if not used[i]

If you can't immediately tell which of these three a new problem is asking, write out the decision tree for a size-3 example by hand before touching code. It takes 30 seconds and prevents building the wrong recursive structure entirely.

10.4 Recognition drills

For each prompt below, identify **state / choices / constraint / base case / undo** before reading the pattern name.

1. "Given a set of distinct integers, return all possible subsets (the power set)." ->

Subsets -- index + path, take/skip, index == n.

2. "Given a collection of numbers that might contain duplicates, return all possible unique permutations." -> **Permutations II** -- `path + used[]` , sort + `not used[i-1]` guard.
3. "Given `n` pairs of parentheses, generate all combinations of well-formed parentheses." -> **Generate Parentheses** -- `open + close + path` , prune `close >= open` .
4. "Given a 2D board and a word, find if the word exists in the grid." -> **Word Search** -- `(r,c,index) + visited` , 4-directional moves, boolean base case.
5. "The n -queens puzzle is the problem of placing n queens on an $n \times n$ chessboard such that no two queens attack each other." -> **N-Queens** -- `row + cols/diag1/diag2` , reject on conflict.
6. "Partition a string such that every substring is a palindrome." -> **Palindrome Partitioning** -- `start + path` , choose next piece end, constraint = palindrome check.
7. "Given digit string 2-9, return all letter combinations the number could represent." -> **Letter Combinations of a Phone Number** -> `index + path` , choices = letters for `digits[index]` .
8. "Solve the Sudoku puzzle by filling the empty cells." -> **Sudoku Solver** -- constraint set per empty cell (row/col/box), boolean short-circuit on first full solution.
9. "Combination Sum, but each number may only be used once, and the solution set must not contain duplicate combinations." -> **Combination Sum II** -- `i+1` (no reuse) + sort + same-level dup guard + `remaining < 0` prune.
10. "Return all ways to partition a string into a sequence of dictionary words, but return the sentences, not just whether it's possible." -> **Word Break II** -- partition shape, constraint = "substring in dictionary," base case collects the joined sentence.

10.5 One-paragraph summary to internalize

For any new problem: identify the **one decision** being made at a single level of the tree (not the whole solution -- just one level). That decision tells you the **choices**. What you'd need to remember to describe "where you are" on the current branch is the **state**. Whether some choices can be rejected outright before recursing is the

constraint/pruning. When the state describes a *complete* answer is the **base case**.

Whatever you mutated to make the choice is exactly what you must **undo** after recursing. Every problem in this guide is that same five-step derivation with different fill-in-the-blanks.

Part 11 -- Interview Q&A

Q: What's the difference between recursion, DFS, and backtracking?

Recursion is just a function calling itself. DFS is a traversal strategy (go deep before wide). Backtracking is DFS/recursion specifically applied to a space of *choices*, where you mutate shared state before recursing and restore it after, so sibling branches start from a clean slate. Every backtracking solution is recursive DFS; not every recursive DFS is backtracking (plain tree traversal isn't, for instance).

Q: How do you approach a new backtracking problem you haven't seen before?

Answer five questions, in order, before writing code: What's my **state**? What are my **choices** from that state? Which choices are **invalid** (pruning)? When is the state a **complete** answer (base case)? What did I mutate that needs to be **undone** after recursing? The code is then a close-to-mechanical translation of those five answers.

Q: Why does `res.append(path.copy())` matter, and when can you skip the `.copy()` ?

`path` is a single mutable object that's `append` ed to and `pop` ped from throughout the whole search. If you store the same reference in `res` without copying, every later mutation retroactively corrupts previously recorded answers -- all entries in `res` end up pointing at the *same* list, whatever its final state happens to be. You can skip the copy only when you immediately convert to an immutable form at the base case (e.g. `"".join(path)` for strings, as in Generate Parentheses and Letter Combinations) -- that conversion already produces a new, independent object.

Q: In Combination Sum, why does choosing a candidate recurse with `backtrack(i, ...)` instead of `backtrack(i+1, ...)` ?

Because reuse is allowed: after choosing `candidates[i]`, it must remain available as a choice on the next call. Advancing to `i+1` on the *choose* branch would silently forbid reusing that candidate, which produces a plausible-looking but incomplete result set (this exact reversal is walked through as a worked correction in Part 3.5). The rule in general: **can I reuse the current choice? yes -> recurse with `i`. no -> recurse with `i+1`.**

Q: How is the `index` in Subsets different from the `start` in Combinations? They're both just integers into the same array.

`index` means "I am deciding about *this one specific* element right now" -- there are always exactly two choices (take/skip), and both branches advance to `index+1`. `start` means "I may choose *any* element at or after this position" -- the number of choices varies with `n - start`, and after choosing element `i` the next call uses `i+1` (or `i`, if reuse is allowed), not `start+1`. Same data type, different role in the decision tree.

Q: How do you avoid duplicate output when the input has duplicate values (Subsets II, Combination Sum II, Permutations II)?

Sort first so equal values are adjacent, then add a same-level-only guard: skip a candidate if it's equal to the previous one **and** the previous one was already considered as an alternative at this exact recursion level (for index-based loops: `i > start` and `nums[i] == nums[i-1]`; for used-array permutations: `not used[i-1]`). The key distinction to say out loud: reusing an equal value at a *deeper* level (e.g. choosing 2, then 2 again) is fine and necessary; reusing it as an *alternative* choice at the *same* level is what produces the duplicate.

Q: Why does Word Search's base case return `True` instead of appending to a results list?

Because the problem only asks "does a valid path exist?" -- a single yes/no answer -- rather than "what are all valid paths?" The boolean propagates back up through `backtrack(...)` or `backtrack(...)` or `...`, so the search can short-circuit the instant one direction succeeds, rather than exhaustively exploring all four from every cell.

Q: What's the actual complexity of backtracking, and how does pruning change it?

The loose worst-case bound is `branching_factor^depth`. Pruning doesn't change that theoretical worst case (an adversarial input can always be built where nothing gets pruned), but rejecting a branch at depth `k` removes an entire subtree of size roughly `b^(depth-k)` -- which is why constraint checks in N-Queens or Sudoku turn a nominally astronomical search space into something that finishes in milliseconds on realistic inputs. See Part 9 for a per-pattern breakdown.

Q: When should you reach for backtracking versus dynamic programming?

If the problem wants **all** valid solutions (or one, but needs to actually construct/return it, like a board or a sequence), backtracking is usually required -- DP typically only gives you a count or an optimum, not the full enumerated set. If overlapping subproblems exist and you only need a count or optimal value (not every solution), DP is usually far more efficient, since it avoids re-exploring identical subproblems. Target Sum (Part 8.2) is a good example of a problem solvable either way: backtracking enumerates every sign assignment directly; a DP reformulation counts them without enumerating.

Q: How would you explain the N-Queens diagonal trick (`row - col` , `row + col`) without memorizing it?

Write out coordinates along one `\` diagonal -- they all share the same `row - col` value (walking down-right, both `row` and `col` increase by 1 each step, so their difference is invariant). Along a `/` diagonal, `row` increases while `col` decreases (or vice versa) by 1 each step, so their *sum* stays invariant. Once you can re-derive that on a whiteboard in 20 seconds, you never need to have memorized it.

Q: A candidate's Combination Sum II solution isn't pruning duplicates correctly -- what's the most common bug?

Usually one of: (1) forgetting to sort first, so equal values aren't adjacent and the `nums[i] == nums[i-1]` check misses them; (2) writing the guard as `i > 0` instead of `i > start`, which incorrectly skips a legitimate *first* choice at a given recursion level just because it matches something chosen at a shallower level; or (3) applying the guard on a *used-array* permutation problem without the `not used[i-1]` half, conflating same-level dedup with the different used-array formulation.